DOCUMENTATION · PROJECT

# Contributing

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?d=CONTRIBUTING. This copy was made on 4 November 2026.

> **Not accepting outside code contributions at the moment.** Bug reports, questions and suggestions are welcome as issues; security problems go through SECURITY.md. The pull-request process below describes how it would work if that changes, and any code accepted would be licensed under the Apache License, Version 2.0 (see LICENSE). Please do not spend time on a pull request without agreeing it with the maintainer first.

When contributing to this repository, please first discuss the change you wish to make via issue, email, or any other method with the owners of this repository before making a change.

Please note we have a code of conduct, please follow it in all your interactions with the project.

## Pull Request Process

1. Ensure any install or build dependencies are removed before the end of the layer when doing a build.
2. Update the README.md with details of changes to the interface, this includes new environment variables, exposed ports, useful file locations and container parameters.
3. Increase the version numbers in any examples files and the README.md to the new version that this Pull Request would represent. The versioning scheme we use is SemVer - see CHANGELOG.md for what's covered by it and this project's pre-1.0 policy.
4. You may merge the Pull Request in once you have the sign-off of two other developers, or if you do not have permission to do that, you may request the second reviewer to merge it for you.

## Code of Conduct

### Our Pledge

In the interest of fostering an open and welcoming environment, we as contributors and maintainers pledge to making participation in our project and our community a harassment-free experience for everyone, regardless of age, body size, disability, ethnicity, gender identity and expression, level of experience, nationality, personal appearance, race, religion, or sexual identity and orientation.

### Our Standards

Examples of behavior that contributes to creating a positive environment include:

- Using welcoming and inclusive language
- Being respectful of differing viewpoints and experiences
- Gracefully accepting constructive criticism
- Focusing on what is best for the community
- Showing empathy towards other community members

Examples of unacceptable behavior by participants include:

- The use of sexualized language or imagery and unwelcome sexual attention or advances
- Trolling, insulting/derogatory comments, and personal or political attacks
- Public or private harassment
- Publishing others' private information, such as a physical or electronic address, without explicit permission
- Other conduct which could reasonably be considered inappropriate in a professional setting

### Our Responsibilities

Project maintainers are responsible for clarifying the standards of acceptable behavior and are expected to take appropriate and fair corrective action in response to any instances of unacceptable behavior.

Project maintainers have the right and responsibility to remove, edit, or reject comments, commits, code, wiki edits, issues, and other contributions that are not aligned to this Code of Conduct, or to ban temporarily or permanently any contributor for other behaviors that they deem inappropriate, threatening, offensive, or harmful.

### Scope

This Code of Conduct applies both within project spaces and in public spaces when an individual is representing the project or its community. Examples of representing a project or community include using an official project e-mail address, posting via an official social media account, or acting as an appointed representative at an online or offline event. Representation of a project may be further defined and clarified by project maintainers.

### Enforcement

Instances of abusive, harassing, or otherwise unacceptable behavior may be reported by contacting the project team at arcswdev@gmail.com. All complaints will be reviewed and investigated and will result in a response that is deemed necessary and appropriate to the circumstances. The project team is obligated to maintain confidentiality with regard to the reporter of an incident. Further details of specific enforcement policies may be posted separately.

Project maintainers who do not follow or enforce the Code of Conduct in good faith may face temporary or permanent repercussions as determined by other members of the project's leadership.

### Attribution

This Code of Conduct is adapted from the Contributor Covenant, version 1.4, available at http://contributor-covenant.org/version/1/4

## Development setup

```shell
git clone https://github.com/AnanthaRajuC/QueryAPIGate.git && cd QueryAPIGate
python -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"
python .github/scripts/fetch_h2_jar.py  # the H2 driver, from Maven Central - only for h2 connections,
or a build

ruff check .
mypy queryapigate
python -m unittest discover -s tests -t .
```

`tests/test_integration.py` runs against real MySQL, PostgreSQL, ClickHouse, H2 and MongoDB servers when the matching `QUERYAPIGATE_IT_*` environment variables are set (see the file header); otherwise those tests are skipped. CI runs them against service containers. `tests/test_sql_guard_fuzz.py` property-tests the SQL guard and parameter binder with Hypothesis; a failure there prints a minimal reproducing example.

`mypy` is configured for gradual typing (`[tool.mypy]` in `pyproject.toml`) - the codebase has no type hints yet, so it only catches genuine static errors, not missing annotations. `coverage run -m unittest discover -s tests -t .` followed by `coverage report` shows local coverage; CI uploads it to Codecov.

**Workflows and the Dockerfile are pinned** (ADR 0018): an action as `owner/repo@<full commit SHA> # vX.Y.Z`, a base image as `name:tag@sha256:<digest>`. `git ls-remote --tags https://github.com/<owner>/<repo>` gives a tag's commit. Dependabot proposes the updates, and `tests/test_supply_chain.py` fails on anything unpinned.

### Console (frontend)

The QueryAPIGate Console - the admin UI at `/console` - lives in `frontend/` (React + TypeScript + Vite; see ADR 0001). **Backend work never needs Node.js:** without a build, `/console` shows a short "not built" page, and everything it does is plain API (`/docs`). To see the UI from a source checkout, build it once (below) or run the Docker image.

To work on it (Node.js 22, see `frontend/.nvmrc`), run the backend as usual, then in a second terminal:

```shell
cd frontend
npm ci
npm run dev        # http://localhost:5173/console/ - API calls are proxied to http://127.0.0.1:5000
npm test           # also: npm run lint, npm run typecheck, npm run format:check
npm run build      # writes queryapigate/console_dist/, served by the backend at /console
```

The Console's TypeScript types are generated from `frontend/openapi.json`, a committed copy of the backend's OpenAPI description. **If you change `queryapigate/openapi.py`, regenerate it** - Python only, no Node needed:

```shell
python frontend/scripts/dump_openapi.py
```

The backend test suite fails while the two disagree, and also checks that every route and method is documented.

End-to-end tests drive the built Console in Chromium against a real server - a throwaway home with the example APIs (`frontend/e2e/`, run in CI on every push):

```shell
cd frontend
npm run build
npx playwright install chromium   # once
npm run e2e                       # set QUERYAPIGATE_BIN=../.venv/bin/queryapigate if it isn't on PATH
```

The README's screenshots (`documentation/screenshots/`) are taken the same way, from a throwaway server seeded with a realistic estate (`frontend/scripts/screenshots.mjs`). Retake them all after a visible change, and add a screen there - keeping the existing file names, which README.md links to:

```shell
cd frontend
npm run build
npm run screenshots               # QUERYAPIGATE_BIN as above; OUT=/some/dir/ to write elsewhere first
```

Many comments in `frontend/src` cite the function of `queryapigate/ui.py` a screen was ported from - the hand-written admin page the Console replaced. It was removed after the migration; read it in git history (`git show <commit>^:queryapigate/ui.py`, where `<commit>` is the one that deleted it).

### Docs site

The docs site (deployed to GitHub Pages) is built from this README and `documentation/` - there is nothing to edit under `docs/` itself, each file there is a one-line include (see `mkdocs.yml`) pulling in the real document. Preview it locally after changing any `.md` file:

```shell
pip install -e ".[docs]"
mkdocs serve
```

### Releasing

1. Bump `__version__` in `queryapigate/__init__.py`, and `version` and `appVersion` in `deploy/helm/queryapigate/Chart.yaml` and the data plane's `version` in `dataplane/Cargo.toml` with it (build it once, `cargo build --manifest-path dataplane/Cargo.toml`, so `Cargo.lock` follows; a test fails if any differ - the data plane pairs only with Python of its own release, ADR 0030), and turn `## [Unreleased]` in CHANGELOG.md into a dated `## [X.Y.Z] - YYYY-MM-DD` section, with what upgraders must know first. `main` takes changes only through a pull request whose required checks pass: commit on a `release/X.Y.Z` branch, open a pull request, and squash-merge it once it's green.

2. Tag the merged commit on `main` `vX.Y.Z` and push the tag: the Release workflow checks the tag against the version and the changelog, builds the Console into the wheel, publishes to PyPI with build provenance, and makes the GitHub Release (the files, their `.intoto.jsonl`, the changelog's section as notes); the Docker workflow publishes the image. Check one: `gh attestation verify <the wheel> --owner AnanthaRajuC`.

3. Add the release's upgrade fixtures (`tests/fixtures/stores/`, BACKLOG #65) from the published package, on a branch with step 4, so every later version is tested against a store it really wrote:

   ```shell
   python -m venv /tmp/qag-X.Y.Z && /tmp/qag-X.Y.Z/bin/pip install
   "queryapigate[server,postgres]==X.Y.Z"
   python tests/fixtures/stores/generate.py /tmp/qag-X.Y.Z/bin/queryapigate
   python tests/fixtures/stores/generate.py --postgres postgresql://postgres:pw@localhost:5432/postgres
   \
     /tmp/qag-X.Y.Z/bin/queryapigate   # a scratch database; needs pg_dump on PATH
   ```

4. Point the upgrade tests at it: set `PREVIOUS_RELEASE: "X.Y.Z"` in `.github/workflows/ci.yml`, so the next version is tested serving beside this one on an upgraded store, and its Helm chart upgraded from this one's on a cluster.

5. Retake the screenshots (`npm run screenshots`, above) - the Console's Help now reads that release's docs.